

MANUAL TESTING ASSESSMENT

WRITEBACK JOURNAL DRAFT

Quality assurance analysis for a custom Excel function that creates draft GL journal batches for manual review.

52

TEST CASES

28

TEST CATEGORIES

32

SPEC GAPS

10

RISK MAP

Submission package

Review document in PDF format plus a separate Excel workbook containing the complete test case suite.

LIGHT EDITION
EXPANDED VIEW

CONTENTS

01	Overview Feature description, signature, parameters and acceptance criteria	CONTEXT
02	Spec Review Ideal specification outline, field contract and risk map	EXPANDED
03	Spec Gaps & Assumptions All 32 identified ambiguities with assumptions and coverage	32 ITEMS
04	AI Reflection Prompts, review process, limitations and manual decisions	PROCESS
05	Test Cases Workbook Separate Excel file designed for filtering and execution	XLSX

52

TEST CASES

28

TEST CATEGORIES

32

SPEC GAPS

10

RISK MAP

◆ User Story

As an ERP user, I want to create draft GL journal entries from Excel so that I can prepare transactions for manual review and posting by an accountant.

◆ Custom Function Description

`WRITEBACKJOURNALDRAFT` is an Excel custom function for creating draft GL journal batches in the ERP directly from a spreadsheet range. Each function defines a single line. Several of those functions placed together on a worksheet can define one GL journal batch. The function never posts the batch - it always lands in the **On Hold** status, and must be reviewed and posted manually inside the ERP system.

● Function Signature

```
=WRITEBACKJOURNALDRAFT(  
  // batch properties  
  connectionName,  
  date,  
  reportingPeriod,  
  batchDescription,  
  branch,  
  ledger,  
  // line properties  
  currency,  
  account,  
  debit,  
  credit,  
  lineDescription  
)
```

● Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
connectionName	string	YES	ERP connection name
transactionDate	date	YES	Date of the journal batch
postPeriod	string	YES	Financial period (e.g. "05-2026")
batchDescription	string	NO	Batch-level description
branch	string	NO	Branch identifier
ledger	string	YES	Target ledger
currency	string	NO	Currency code; defaults to ledger currency
account	string	YES	GL account per line
debit	number	NO	Debit amount per line
credit	number	NO	Credit amount per line
lineDescription	string	NO	Line description

◆ Acceptance Criteria

Batch created on hold

The resulting batch in the ERP always has status **On Hold**; no release or post action is triggered.

Balance validation

Total debits must equal total credits across all lines; imbalanced input returns "Journal is not balanced: debits X.XX ≠ credits Y.YY" error.

At least one line required

An empty line range returns "At least one transaction line is required" .

Return value

Returns "Draft batch #JE-00087 created" on success.

□ Glossary

Batch

A batch is a group of journal lines (transactions) that have the same transaction date, ledger, and batch description.

Reporting Period

A reporting period is a predefined accounting period where the transaction date belongs (i.e. "Month Ending 31 March 2027").

02 SPEC REVIEW

Expanded ideal specification, field contract and financial risk map

◇ Ideal Specification - What Is Missing

A complete specification for this feature should include all of the following sections. Sections absent from the current spec are the source of the 32 gaps documented in the Spec Gaps page.

01 Business goal and user story

Defines who uses the feature and why a draft is created instead of an immediately posted transaction.

EXAMPLE

As an ERP user, I prepare GL journals in Excel for accountant review; writeback always creates an unposted draft.

02 Definitions: batch, line, ledger, postPeriod, debit, credit, On Hold

Removes interpretation differences between a tester, developer and accountant.

EXAMPLE

A batch is a group of lines with one grouping key; *On Hold* means created but not released or posted.

03 Processing model: explicit Writeback vs. formula recalculation

Clarifies the event that sends data to ERP, which is essential for duplicate prevention.

EXAMPLE

Recalculating or reopening the workbook must not create a batch; only the explicit Writeback action sends the request.

04 Excel input rules and accepted date/number formats

Defines what is valid before ERP validation and avoids locale-related defects.

EXAMPLE

`DATE(2026,5,19)` is accepted and sent as `2026-05-19`; ambiguous text dates are rejected.

05 Normalized REST/OData/API payload and response contract

States exactly what the add-in sends and what success or failure response it can display.

EXAMPLE

Success response contains `batchNumber`, `status=OnHold` and `correlationId`.

06 ERP field mapping and validation responsibilities

Shows where each Excel value lands and whether Excel or ERP owns validation.

EXAMPLE

`ledger` maps to ERP Ledger ID; existence and access permissions are checked server-side.

07 Batch grouping rules - full grouping key definition

Prevents lines for different entities from being silently combined into one journal batch.

EXAMPLE

Lines are grouped only when connection, date, period, branch, ledger, currency and description all match.

08 Line-level debit/credit rules - mutual exclusion

Defines valid accounting shape of a single line.

EXAMPLE

Each line must contain a positive debit or a positive credit, never both and never neither.

09 Balance validation scope - per batch, not global

Makes clear that one imbalanced group cannot be hidden by another compensating group.

EXAMPLE

Batch A debit 100 / credit 90 is rejected even if Batch B has debit 90 / credit 100.

10 Idempotency and duplicate prevention

Protects financial data when a user retries after a delay or lost response.

EXAMPLE

The same operation ID returns the original batch number and never creates a second batch.

11 Atomicity and partial failure behavior

Specifies whether all lines must be saved together and how failures roll back.

EXAMPLE

If line 4 has an invalid account, zero header or line records remain in ERP.

12 Timeout/retry behavior and safe error messages

Avoids misleading success or blind retries when ERP state is unknown.

EXAMPLE

On timeout, display an operation ID and advise status lookup; retry uses the same idempotency key.

13 Permissions model and access control

Determines which users may write and which ledgers or branches they may target.

EXAMPLE

A user without Create Draft Journal permission receives an authorization error and no ERP record.

14 Telemetry, audit trail, and correlation ID

Supports investigation of financial operations without exposing sensitive content.

EXAMPLE

Audit stores user, timestamp, operation ID, batch number and outcome, excluding credentials.

15 Cross-platform support: Windows, macOS, Excel Web

States environments in which the function is expected to behave consistently.

EXAMPLE

Windows and macOS desktop are supported; Excel Web either works identically or shows a supported-platform message.

16 Error codes, messages, and validation priority order

Makes failures predictable when a row violates several rules at once.

EXAMPLE

Required-field errors appear before balance checks and identify the field and Excel row.

17 Field length limits and numeric precision rules

Defines boundaries for text and monetary values so no rounding or truncation is silent.

EXAMPLE

Amounts support two decimal places; over-precision is rejected rather than silently rounded.

18 Out of scope and known limitations

Prevents reviewers from expecting behavior the feature intentionally does not provide.

EXAMPLE

Posting, releasing, editing existing batches and exchange-rate calculation are outside this function.

❑ Ideal Field Specification

Proposed field contract: how each parameter should be defined in a complete, production-ready feature specification. It connects Excel input, normalised API data, ERP mapping, validation, defaults, and limits.

01 / INPUT

Spreadsheet values

Fields entered by the user: dates, ledger, account, currency, descriptions and debit/credit amounts.

02 / CONTROL

Safe processing

System-generated operation and line identifiers used for duplicate prevention, tracing and safe retry.

03 / OUTPUT

Observable result

Success/error response, batch status, ERP lookup and audit evidence needed to confirm the writeback result.

QA REVIEW

Good foundation, but not sufficient for test execution without clarification.

Source / status

Mark each rule as original requirement, agreed clarification, or proposed control.

Error contract

Define code, message, affected row/field/group and validation order.

Validation ownership

State whether Excel add-in, API or ERP rejects each invalid value.

Processing rules

Identify grouping-key fields, idempotency scope and audit/security handling.

Requirement boundary: the user-input rows represent fields described in the task. `externalLineId`, `idempotencyKey` and `correlationId` are proposed reliability controls, not confirmed source requirements.

FIELD	SCOPE	REQUIRED	EXCEL FORMAT	API FORMAT	ERP MAPPING	VALIDATION RULE	DEFAULT	LIMIT
connectionName	Input / system	YES	Text - connection name	string	ERP connection selector	Must exist and be accessible	none	255 chars
transactionDate	Batch	YES	Excel date or <code>DATE(Y,M,D)</code>	YYYY-MM-DD	Transaction Date	Valid date; must belong to postPeriod	none	ISO date
postPeriod	Batch	YES	Text period code, e.g. <code>05-2026</code>	string / periodId	Financial Period	Exists in ERP and is open (not closed/locked)	none	Format defined by ERP
batchDescription	Batch	NO	Text	string	Batch Description	Safe text; part of grouping key	empty / null	255 chars
branch	Batch	NO / DEPENDS	Branch code	branch code / id	Branch	If provided: exists and accessible; part of grouping key	default / empty	30–50 chars
ledger	Batch	YES	Ledger code / name / ID - must be specified	ledger code / id	Ledger	Active and accessible ERP ledger	none	30–50 chars
currency	Batch / Line	NO	Currency code	currency code	Currency	If provided: validate the entered currency; if empty: use the selected ledger currency	Selected ledger currency, only when this field is empty	3 chars (ISO 4217)
account	Line	YES	Account code	account code / id	GL Account	Active and accessible in chart of accounts	none	30–50 chars

FIELD	SCOPE	REQUIRED	EXCEL FORMAT	API FORMAT	ERP MAPPING	VALIDATION RULE	DEFAULT	LIMIT
debit	Line	Conditional	Excel number	decimal	Debit Amount	> 0; valid precision; only if credit is empty	null	decimal(19,4)
credit	Line	Conditional	Excel number	decimal	Credit Amount	> 0; valid precision; only if debit is empty	null	decimal(19,4)
lineDescription	Line	NO	Text	string	Line Description	Safe text; no script or markup execution	empty / null	255 chars
externalLineId	Line / system	Generated	Hidden / generated	string	Trace / source line	Unique within operation; links ERP line back to Excel row	auto-generated	36-255 chars
idempotencyKey	Operation / system	Generated	Hidden / sent as request header	string / uuid	Writeback operation	Unique per operation; prevents duplicate batches on retry	auto-generated	36-255 chars
correlationId	Operation / system	Generated	Hidden / generated	string / uuid	Trace / audit	Unique per operation; links Excel, API logs, telemetry, ERP audit	auto-generated	36-255 chars

Contracts still needed before QA sign-off

Response contract

Specify success payload and visible result, for example batch number, **On Hold** status and correlation ID.

Error catalogue

Specify messages/codes for required fields, balance, invalid master data, permissions and timeout states.

Grouping and atomicity

List the full batch key and whether one invalid group blocks all groups or only that group.

Retry and audit evidence

Define duplicate protection, safe retry outcome and what evidence is available in ERP/logs.

◆ Risk Map - Financial Impact

● Duplicate batch creation

Excel recalculation, workbook reopen, or timeout retry creates multiple identical GL journal drafts - a critical financial data integrity risk.

● Wrong status or auto-posting

Batch bypasses "On Hold" and gets posted/released automatically - journal entries appear in financial reports without accountant review.

● Wrong balance validation scope

Global-range balance check masks imbalanced individual batches - incorrect financial data passes silently.

● Partial batch creation (atomicity)

One invalid line leaves a partial batch in ERP - an imbalanced orphaned journal entry that is difficult to detect and reverse.

● Wrong ledger / account / period

Financial data posted to incorrect ledger or period corrupts reporting and may require audit-trail investigation to reverse.

- **Date/locale conversion errors**

Ambiguous text dates (05/06 = May 6 vs June 5) silently produce wrong accounting dates across platforms.

- **Permission bypass**

Unauthorized financial writes if ERP-side permissions are not enforced server-side.

- **Missing traceability**

No correlation ID or audit log entry means support cannot investigate ERP/API failures after they occur.

- **Cross-platform inconsistency**

Different date or decimal behavior between Windows / macOS / Excel Web produces incorrect journal data depending on the user's environment.

- **Script / markup injection via text fields**

Unsanitized batchDescription or lineDescription could execute in ERP UI - XSS or formula injection risk.

4

CRITICAL

21

HIGH

7

MEDIUM

32

TOTAL GAPS

G-01

HIGH

Parameter Names Are Inconsistent

WHAT IS UNCLEAR

The function signature uses `date` and `reportingPeriod`, but the parameter table uses `transactionDate` and `postPeriod` for the same fields. It is not clear which names should be used in documentation, error messages, and test cases.

WHY IT MATTERS

If different names are used in different places, users and testers may not understand which field failed validation.

ASSUMPTION

Use the parameter table names for testing: `transactionDate` and `postPeriod`. The product spec should choose one final naming standard.

TEST COVERAGE

Validation cases for required fields and error messages.

G-02

CRITICAL

It Is Not Clear When Data Is Sent To ERP

WHAT IS UNCLEAR

The spec does not clearly say whether data is sent to ERP when the formula recalculates, or only when the user starts a Writeback action.

WHY IT MATTERS

Excel can recalculate when the file opens, when a cell changes, or when the user presses F9. If recalculation writes to ERP, duplicate journal drafts may be created.

ASSUMPTION

Data should be sent to ERP only when the user starts an explicit Writeback action. Normal Excel recalculation must not create or update anything in ERP.

TEST COVERAGE

Excel recalculation, workbook reopen, and duplicate prevention cases.

G-03

CRITICAL

Idempotency Is Not Defined

WHAT IS UNCLEAR

The spec does not describe how the system prevents duplicate batches when the same operation is retried after timeout, workbook reopen, or user retry.

WHY IT MATTERS

Without idempotency, one network timeout can lead to two identical journal drafts in ERP.

ASSUMPTION

The backend should use an idempotency key or operation ID. Retrying the same operation should return the original batch, not create a new one.

TEST COVERAGE

Retry after timeout and duplicate prevention cases.

G-04

HIGH

Batch Grouping Rules Are Not Fully Clear

WHAT IS UNCLEAR

The spec says one batch is created from lines with the same transaction date, ledger, and batch description. It does not say whether `connectionName`, `postPeriod`, `branch`, or `currency` also split lines into different batches.

WHY IT MATTERS

If grouping is unclear, lines from different periods, branches, or currencies may be placed into the wrong batch.

ASSUMPTION

A batch should be created only from lines that have the same `connectionName`, `transactionDate`, `postPeriod`, `batchDescription`, `branch`, `ledger`, and `currency`.

TEST COVERAGE

Batch grouping and per-batch balance cases.

G-05

CRITICAL

Balance Check Scope Is Not Clear

WHAT IS UNCLEAR

The spec says balance is checked across all lines, but it does not say whether this means all selected lines globally or each grouped batch separately.

WHY IT MATTERS

Two wrong batches can look balanced if their totals cancel each other globally. That is not safe for accounting.

ASSUMPTION

Debit and credit totals should be checked separately for each batch.

TEST COVERAGE

Per-batch balance cases.

G-06

HIGH

Batch Or Voucher Balance Level Is Not Clear

WHAT IS UNCLEAR

Some ERP systems check balance at batch level, while others may check it at voucher or journal entry level. The spec does not say which level is correct.

WHY IT MATTERS

Testers need to know the exact level where debit and credit must match.

ASSUMPTION

Use batch-level balance unless the ERP has a lower-level voucher rule that is documented.

TEST COVERAGE

Balance and ERP mapping cases.

G-07

HIGH

Debit And Credit Rules Are Not Fully Defined

WHAT IS UNCLEAR

Debit and credit are optional in the field table, but the spec does not say whether one line can have both debit and credit, or neither of them.

WHY IT MATTERS

Ambiguous debit and credit rules can create meaningless or wrong journal lines.

ASSUMPTION

Each line should have exactly one side: either debit greater than zero or credit greater than zero. A line must not have both and must not have neither.

TEST COVERAGE

Debit and credit validation cases.

G-08

HIGH

Minimum Valid Line Rule Is Incomplete

WHAT IS UNCLEAR

The spec says at least one transaction line is required, but one line alone cannot create a balanced journal batch.

WHY IT MATTERS

A single debit line or a single credit line can never pass balance validation.

ASSUMPTION

A valid batch should have at least one debit line and at least one credit line.

TEST COVERAGE

Minimum line count and balance cases.

G-09

MEDIUM

Date Format Is Not Clearly Defined

WHAT IS UNCLEAR

Excel dates are locale-sensitive. A text date like `05/06/2026` can mean different dates in different regions.

WHY IT MATTERS

Silent date conversion may create a journal in the wrong accounting date.

ASSUMPTION

Accept real Excel date values or `DATE()` formulas and normalize them to YYYY-MM-DD. Text dates should be rejected unless a locale rule is documented.

TEST COVERAGE

Date format and cross-platform date cases.

G-10

HIGH

Reporting Period Is Not Clearly Defined

WHAT IS UNCLEAR

The spec uses `reportingPeriod` or `postPeriod`, but it does not explain whether this is a period code, a month, a date range, or an ERP financial period.

WHY IT MATTERS

A period is not just one date. It has a start and end date, and the system must know them.

ASSUMPTION

`postPeriod` should be an ERP financial period, for example `05-2026`. The system should validate that `transactionDate` is inside this period.

TEST COVERAGE

Period validation and date-period consistency cases.

G-11

HIGH

Period Start And End Dates Are Not Described

WHAT IS UNCLEAR

The spec does not say how the system knows the start and end date of the selected period.

WHY IT MATTERS

Without period boundaries, the system cannot safely check whether a transaction date belongs to the selected period.

ASSUMPTION

The system should get period start and end dates from ERP master data and validate `transactionDate` against them.

TEST COVERAGE

Date-period validation cases.

G-12

HIGH

Closed Period Behavior Is Not Defined

WHAT IS UNCLEAR

The spec does not say what happens if the selected `postPeriod` is closed or locked in ERP.

WHY IT MATTERS

Writing financial data to a closed period can break accounting controls.

ASSUMPTION

Creating a draft in a closed or locked period should be rejected with a clear Excel error.

TEST COVERAGE

Closed period cases.

G-13

HIGH

Allowed Ledger Values Are Not Listed

WHAT IS UNCLEAR

The spec says ledger is required, but it does not say which values are valid. It is unclear whether the user should enter ledger code, ledger name, or ERP internal ID.

WHY IT MATTERS

Users may enter the wrong value if the accepted ledger format is not clear.

ASSUMPTION

Ledger code is the safest user-facing assumption unless the product confirms another value.

TEST COVERAGE

Ledger validation and permission cases.

G-14

HIGH

Account Identifier Is Not Defined

WHAT IS UNCLEAR

The spec says account is required, but it does not say whether the user should enter account code, account name, or ERP internal ID.

WHY IT MATTERS

Account names can be duplicated or translated. Internal IDs may not be visible to business users.

ASSUMPTION

The user should enter the ERP account code. The system should validate that the account exists, is active, and is available to the current user.

TEST COVERAGE

Account validation cases.

G-15

HIGH

Account Validation Rules Are Missing

WHAT IS UNCLEAR

The spec does not describe how invalid, inactive, restricted, or missing accounts should be handled.

WHY IT MATTERS

Wrong accounts lead to wrong financial data and are hard to fix later.

ASSUMPTION

The system should reject accounts that do not exist, are inactive, or are not available to the user.

TEST COVERAGE

Account required, invalid account, inactive account, and restricted account cases.

G-16

HIGH

Allowed Currencies For Each Ledger Are Not Defined

WHAT IS UNCLEAR

The spec says currency can default from the ledger, but it does not say which currencies are allowed for each ledger.

WHY IT MATTERS

A ledger may not allow every currency. A wrong ledger and currency combination should not create a batch.

ASSUMPTION

The system should validate currency against the selected ledger. If currency is empty, the system should use the currency configured for that ledger.

TEST COVERAGE

Currency and ledger combination cases.

G-17

HIGH

Multi-Currency Behavior Is Not Defined

WHAT IS UNCLEAR

The spec does not say whether one batch can include lines with different currencies.

WHY IT MATTERS

Balance between different currencies is not meaningful without conversion rules.

ASSUMPTION

All lines in one batch should use the same currency unless ERP conversion rules are clearly documented.

TEST COVERAGE

Currency validation and grouping cases.

G-18

MEDIUM

Branch Behavior Is Not Defined

WHAT IS UNCLEAR

Branch is optional, but the spec does not say whether branch is batch-level or line-level, whether it is validated, or whether it affects grouping.

WHY IT MATTERS

Lines from different branches should not be mixed unless the ERP explicitly allows it.

ASSUMPTION

Treat branch as a batch-level field. If it is provided, validate it and include it in the grouping key.

TEST COVERAGE

Branch validation and grouping cases.

G-19

CRITICAL

Atomicity Behavior Is Missing

WHAT IS UNCLEAR

The spec does not say what happens if one line fails validation or ERP fails in the middle of creation.

WHY IT MATTERS

Partial journal creation can leave wrong or incomplete financial data.

ASSUMPTION

Use all-or-nothing behavior. If any line or batch fails, nothing should be written to ERP.

TEST COVERAGE

Atomicity and partial failure cases.

G-20

HIGH

Partial Success Across Multiple Batches Is Not Defined

WHAT IS UNCLEAR

The selected range may create several batches. The spec does not say what happens if one batch is valid and another one is invalid.

WHY IT MATTERS

Partial success can confuse users and make reconciliation harder.

ASSUMPTION

The safest default is all-or-nothing for the whole Writeback operation.

TEST COVERAGE

Mixed valid and invalid group cases.

G-21

HIGH

Permission Model Is Not Described

WHAT IS UNCLEAR

The spec does not define which ERP permissions are needed to create draft journals, use ledgers, use branches, or use accounts.

WHY IT MATTERS

Financial write access must not depend only on Excel-side checks.

ASSUMPTION

ERP permissions should be enforced server-side. If access is missing, Excel should show a clear permission error and no batch should be created.

TEST COVERAGE

Permission cases.

G-22

HIGH

User-Facing Excel Error Behavior Is Not Described

WHAT IS UNCLEAR

The spec does not say where and how errors are shown in Excel. It is unclear whether errors appear in the formula cell, task pane, notification, or result area.

WHY IT MATTERS

Excel is the main user interface. Users must understand what failed and where.

ASSUMPTION

Excel should show a clear user-visible error with the affected field and row or batch group.

TEST COVERAGE

Required field, invalid master data, balance, and technical error cases.

G-23

MEDIUM

Success Result Per Excel Row Is Not Clear

WHAT IS UNCLEAR

The spec says the function returns a message like `Draft batch #JE-00007 created`, but it does not say what happens when several Excel rows are grouped into one batch.

WHY IT MATTERS

Users need to understand which rows were included in the created batch.

ASSUMPTION

All rows included in the same batch should show the same batch number, or Excel should show one clear grouped result with affected rows.

TEST COVERAGE

Success result and grouped batch cases.

G-24

HIGH

Selected Excel Range Behavior Is Not Clear

WHAT IS UNCLEAR

The spec does not explain how the add-in decides which Excel rows are included in Writeback.

WHY IT MATTERS

Workbooks can contain empty rows, hidden rows, filtered rows, copied formulas, and extra data outside the selected range.

ASSUMPTION

Only the selected or configured range should be processed. Empty, hidden, filtered, and invalid rows should follow documented rules.

TEST COVERAGE

Selected range processing cases.

G-25

HIGH

Cross-Platform Excel Behavior Is Not Defined

WHAT IS UNCLEAR

The spec does not describe expected behavior in Excel for Windows, Excel for macOS, and Excel Web.

WHY IT MATTERS

Dates, decimals, add-in behavior, and formula recalculation can differ by platform.

ASSUMPTION

Behavior should be the same across supported platforms, or limitations should be clearly documented.

TEST COVERAGE

Cross-platform cases.

G-26

HIGH

API Contract Is Not Described

WHAT IS UNCLEAR

The spec does not define the API payload, response schema, field mapping, or error code mapping between Excel, backend, and ERP.

WHY IT MATTERS

Without a contract, API tests are based on guesses instead of confirmed behavior.

ASSUMPTION

The product should document request fields, response fields, required fields, data types, and error format.

TEST COVERAGE

API contract and request/response mapping cases.

G-27

HIGH

Audit Trail Is Not Specified

WHAT IS UNCLEAR

The spec does not say what is recorded in the audit trail for successful and failed Writeback operations.

WHY IT MATTERS

Support needs traceability to investigate duplicates, failed operations, and wrong ERP data.

ASSUMPTION

Each operation should be auditable: user, timestamp, source, result, error code, and created batch number when available.

TEST COVERAGE

Audit trail cases.

G-28

MEDIUM

Error Alerting Is Not Described

WHAT IS UNCLEAR

The spec does not say whether repeated or critical failures create internal alerts (e.g. in Sentry, Slack, or monitoring tools).

WHY IT MATTERS

Teams should know about production failures before many users report them.

ASSUMPTION

Critical failures such as ERP unavailable, timeout, or repeated Writeback failures should trigger an internal alert.

TEST COVERAGE

Monitoring and technical failure cases.

G-29

MEDIUM

Field Limits And Numeric Precision Are Not Specified

WHAT IS UNCLEAR

The spec does not define maximum lengths for text fields or numeric precision for debit and credit.

WHY IT MATTERS

Undefined limits can cause truncation, overflow, rounding issues, or ERP errors.

ASSUMPTION

Define limits for each text field and amount field. Reject values that exceed limits or document safe truncation rules.

TEST COVERAGE

Boundary value cases.

G-30

MEDIUM

Sensitive Data Logging Rules Are Not Defined

WHAT IS UNCLEAR

The spec does not say which request data can be logged and which data must be masked.

WHY IT MATTERS

Logs must help support support without leaking secrets.

ASSUMPTION

Logs should include useful support data but must not include tokens, passwords, or sensitive connection details.

TEST COVERAGE

Telemetry and log safety cases.

G-31

MEDIUM

Preview Before Writeback Is Not Described

WHAT IS UNCLEAR

The spec does not say whether the user can preview the batch before it is created in ERP.

WHY IT MATTERS

This is a financial write operation. A preview helps users catch mistakes before sending data to ERP.

ASSUMPTION

If preview exists, it should show grouped batches, debit total, credit total, ledger, period, currency, and line count before final submission.

TEST COVERAGE

Preview or pre-submit validation cases if the product supports this feature.

G-32

HIGH

Duplicate Business Data Rule Is Not Defined

WHAT IS UNCLEAR

The spec does not say what happens if the user sends the same journal data again as a new operation (new idempotency key, same business content).

WHY IT MATTERS

Idempotency protects retries, but it does not answer whether the same journal can be submitted again intentionally or by mistake.

ASSUMPTION

The product should define whether duplicate business data is allowed, blocked, or shown as a warning.

TEST COVERAGE

Duplicate business data cases.

04 AI REFLECTION

Transparent account of assisted work and human review

01 // SECTION 01 Input Materials Used During the Analysis

Several sources fed the analysis. The spec defined what the function should do; my research and notes added context, open questions, and edge cases that the spec did not cover.

01. The original WRITEBACKJOURNALDRAFT specification
02. My own research into ERP systems, GL journals, writeback flows, and financial terminology
03. Google Docs notes - questions, draft ideas, and risks I spotted while reading the spec
04. Miro board - end-to-end flow, unclear behaviors, edge cases, and missing rules
05. AI tools used during research and drafting: ChatGPT, Claude, and Codex
06. AI-assisted explanations of how similar systems may work under the hood
07. My own QA thinking on Excel behavior, ERP side effects, financial risk, and integration edge cases

02 // SECTION 02 How I Used AI

Honestly, AI saved me the mechanical first-draft work and let me start reacting to something concrete instead of a blank page. I leaned on it most for these things:

WHAT I LEANED ON AI FOR

01. Getting up to speed on a domain I didn't know well - ERP, GL journals, ledgers, periods, debit/credit
02. Seeing the feature as an end-to-end flow
03. Brainstorming test categories and surfacing early risks before I wrote anything
04. Drafting a first version of the cases and gap descriptions in bulk
05. Merging near-duplicates and making the suite leaner
06. Improving wording and turning raw notes into a readable artifact - including phrasing my thinking in clear English

I didn't take any output at face value: each result was kept, rewritten, or thrown out. AI just gave me something to react to. Concretely, it saved up to an hour of boilerplate - time that went into the parts that actually needed thinking.

// PROMPT LOG

FIVE PROMPTS, IN ORDER OF USE

Domain & System Understanding

> PROMPT USED

I need to test a custom Excel function called WRITEBACKJOURNALDRAFT.

Before writing test cases, explain how similar Excel-to-ERP writeback systems usually work. Explain ERP, GL journal batch, journal line, ledger, account, postPeriod, debit, and credit in simple terms.

Show the possible flow: Excel - add-in - API - ERP - draft batch - result.

What risks should a QA engineer think about before writing test cases?

+ WHAT WORKED

- Made me see the feature as a system flow, not just an Excel formula
- Explained ERP terms in plain language - ledger, account, period, journal line
- Separated user-entered fields from generated technical fields
- Surfaced early risks: duplicate creation, wrong status, timeout and retry

- WHAT FELL SHORT

- Explained generic ERP concepts without tying them to the spec
- Presented some assumptions as if they were confirmed behavior
- Needed extra prompting to move from theory to testability

> WHAT I DID WITH IT

- Tied the explanations back to the actual WRITEBACKJOURNALDRAFT fields
- Turned unclear points into gap entries instead of accepting them
- Kept the research as background context, not as final spec

Initial Test Brainstorming

> PROMPT USED

I'm a QA engineer writing test cases for WRITEBACKJOURNALDRAFT.

Each call defines one journal line. Multiple calls form one batch.
The batch must always be On Hold. It must not post automatically.

Here is the full spec: [pasted]

Give me a complete list of test categories and flag ambiguities, missing rules, or risky assumptions.

Focus on: batch grouping, balance, required fields, ERP side effects, idempotency, retry/timeout, permissions, audit, errors, cross-platform.

+ WHAT WORKED

- Mapped the main areas quickly: happy path, validation, balance, status, errors
- Spotted the naming mismatch (**date/reportingPeriod** vs **transactionDate/postPeriod**)
- Suggested decimal-precision risks - important for financial data
- Treated the function as a system integration, not just a formula

- WHAT FELL SHORT

- Took "at least one line required" literally - a single debit can't balance
- Didn't challenge the grouping key (branch? currency? postPeriod?)
- Assumed all cells return the same batch number - the spec doesn't confirm it
- Some suggested cases were too generic, with no concrete test data

> WHAT I DID WITH IT

- Added gap analysis for grouping, return-value scope, and range behavior
- Rewrote vague assumptions as explicit gaps linked to test coverage
- Simplified the language so the gaps are useful to QA, devs, and product

Formal Test-Case Generation

> PROMPT USED

You are a senior QA engineer with strong test-design background.
Write formal test cases for WRITEBACKJOURNALDRAFT.

Format:

TC-ID - Category - Title - Preconditions - Test Data - Steps - Expected Result.

Constraints:

- Simple English. No Priority or Severity.
- Numbered lists for Preconditions, Steps, Expected Result.
- Preconditions specific to each case, not a shared header.
- Group duplicates into one parameterized case.
- Use concrete test data (actual values), not "valid input".

Apply standard test-design techniques explicitly:

- Equivalence partitioning and boundary value analysis for amounts, dates, periods, and text-length limits.
- Decision tables for required-field combinations and debit/credit/account mutual exclusion.
- State transitions around batch status (On Hold lifecycle, no auto-post, no auto-release).
- Pairwise combinations when multiple optional fields interact.
- Error guessing for locale, timezone, decimal separator, Excel-specific quirks.
- Negative cases for every validation rule, not just happy paths.

Treat the feature as an end-to-end integration, not a formula. Cover:

- Excel - Add-in - API - ERP - response back to Excel.
- Idempotency and retry safety (no duplicate batches on retry).
- Atomicity for multi-line operations (all-or-nothing).
- Auditability, traceability, correlation IDs.
- Permissions and master-data scoping (ledger, branch, account).
- ERP side effects: status, totals, account balances.
- Sensitive data: never leak secrets to logs, Excel, or responses.

Think like a product analyst too:

- What does the business actually risk if this case fails?
- Which user persona triggers this path (accountant, controller, auditor, support)?
- Map each case to an acceptance criterion or business rule.

+ WHAT WORKED

- Produced a clean, structured suite quickly
- Covered the main flows: creation, On Hold, no auto-post, required fields, balance, dedupe
- Consistent structure across all cases with separate columns

- WHAT FELL SHORT

- First version had too many near-duplicates - needed manual merging
- Some preconditions were still vague ("valid ERP data exists")
- Verification steps were mixed into Steps instead of Expected Result

> WHAT I DID WITH IT

- Merged near-duplicates into grouped scenarios
- Rewrote preconditions to be specific per case
- Moved all verification into Expected Result; removed Priority/Severity

Gap-Analysis Improvement

> PROMPT USED

You are a senior QA analyst doing a requirements review on the spec.
Goal: produce a gap-and-assumption analysis that QA, devs, and product can act on without further clarification.

Format (one entry per gap):

Gap - What is unclear - Assumption - Why it matters - Test coverage.

Constraints:

- Simple English, no over-explanation.
- Each gap is one concrete, testable concern (not a vague worry).
- Mark severity: Critical / High / Medium, based on financial and business risk, not engineering effort.
- Map each gap to specific test categories or TC-IDs when possible.
- Do not present assumptions as confirmed facts. Phrase as "Assumption: ... unless the spec confirms otherwise."

Look for gaps across these dimensions:

- Functional rules left implicit: grouping keys, field normalization, default values, idempotency, atomicity.
- Validation order and ownership: Excel vs Add-in vs Backend vs ERP.
- Error handling: messages, surface (Excel vs logs), recovery paths, user-facing wording.
- Integration boundaries: API contract, response mapping, timeout, retry, partial success.
- Non-functional: limits, decimal precision, concurrency.
- Security and privacy: permissions, sensitive data in logs, injection-safe text fields, formula execution risk.
- Auditability: correlation IDs, who-did-what-when, alerting on repeated failures.
- Excel-specific risk: workbook reopen, locale and date format.
- Edge cases: empty/blank ranges, blank rows inside range, mixed valid/invalid groups in one selection.

For each gap also capture:

- Stakeholders affected (user, accountant, support, dev, product).
- What the business loses if the assumption is wrong.
- A concrete test scenario that would expose the gap.

Add your own gaps if something important is missing from my Miro notes or from the spec itself.

+ WHAT WORKED

- Restructured the gaps into a cleaner, more readable format
- Connected each gap to practical test coverage
- Added missing areas: Excel error behavior, alerting, range edge cases, duplicate business data, allowed currencies per ledger, partial success, validation order, and sensitive data in logs

- WHAT FELL SHORT

- Sometimes over-explained in technical wording that added no value
- Still framed a few assumptions as confirmed facts

> WHAT I DID WITH IT

- Removed wording that didn't help testing or requirement clarity
- Kept each gap focused on business risk, not just a technical detail
- Removed assumptions that were presented as facts

Field Specification Review

> PROMPT USED

Build the field-level specification the way it should look in a production-ready spec. The Parameters table in the task gives only Type, Required, and a short Description - extend it into a real field contract a developer and tester can both use.

Fields (from the task spec):
connectionName, transactionDate, postPeriod, batchDescription, branch, ledger, currency, account, debit, credit, lineDescription.

Produce a single table with these columns:
Field - Scope (Input/system, Batch, Line) - Required - Excel Format - API Format - ERP Mapping - Validation Rule - Default - Limit.

Then, in a "QA review" block under the table, call out what the task still leaves open. Group findings by:

- Source / status: which rules are from the task, which are proposed clarifications, which are proposed controls.
- Validation ownership: who rejects each invalid value - Excel add-in, API, or ERP.
- Error contract: code, message, affected row/field/group, and the order in which validations fire.
- Processing rules: grouping-key fields, idempotency scope, audit and security handling.

Separately propose a small set of reliability fields the task does not include but a real implementation would need. Choose them yourself based on what the writeback flow actually requires (duplicate prevention, retry safety, support traceability, line-level recovery, etc.). For each proposed field state the concrete failure mode it prevents. Mark every such field as a proposed control, not a source requirement.

Anything the task does not state is an assumption, not a fact.
Phrase as "Assumption: ... unless the spec confirms otherwise."
Open items that fail this rule should end up on the Spec Gaps page using the Gap / What is unclear / Assumption / Why it matters / Test coverage structure.

+ WHAT WORKED

- Produced the full Field / Scope / Required / Excel Format / API Format / ERP Mapping / Validation Rule / Default / Limit table used on the Spec Review page
- Flagged that **ledger** and **account** shouldn't accept code/name/ID interchangeably - one canonical form is needed
- Surfaced the four QA-review gaps used on the page: Source/status, Validation ownership, Error contract, Processing rules
- Suggested reliability fields the task doesn't include (externalLineId, idempotencyKey, correlationId) and marked them as proposed controls, not source requirements

- WHAT FELL SHORT

- Mixed proposed controls into the source-requirement rows - had to split them out manually for the Spec Review note
- Left some limits and defaults as if confirmed, when they were only assumptions
- Open items weren't routed to the Spec Gaps page format on the first pass; had to ask again for the Gap/Unclear/Assumption/Why/Coverage structure

> WHAT I DID WITH IT

- Moved externalLineId / idempotencyKey / correlationId into the explicit "proposed reliability controls" note on the Spec Review page
- Marked every unconfirmed limit and default as an assumption, not a fact
- Routed every remaining open item into the Spec Gaps page using the Gap / What is unclear / Assumption / Why it matters / Test coverage structure

08

// SECTION 08

Where AI Was Not Enough

ACROSS EVERY PROMPT, AI WAS CONSISTENTLY WEAKER AT

- 01. Challenging the spec unless explicitly told to
- 02. Detecting hidden product risks without direct guidance
- 03. Avoiding assumptions when the spec left something open
- 04. Deciding which cases to merge versus keep separate
- 05. Treating Excel-triggered side effects as financial risk, not a UI inconvenience

The pattern: AI reads a specification charitably. It fills gaps with plausible assumptions instead of flagging them as risks. The most valuable part of this task was the opposite instinct - reading carefully and asking, every time: does this actually make sense?

09

// SECTION 09

Why I Decided to Create a Website

As the analysis grew, one flat document became too overloaded. There were test cases, gaps, assumptions, field specifications, validation rules, AI usage notes, and review comments.

Because of that, I decided that a small structured website would be a better format. It would let the reader switch between sections instead of scrolling through one long document.

Looking back now, there are things I would still rewrite with more time. Either way, I really enjoyed working in a new domain. Thank you.

// SEPARATE DELIVERABLE

TEST CASES WORKBOOK

The full test case suite is provided as a separate Excel workbook so that areas, test data, preconditions, steps and expected results remain easy to filter, scan and execute.

XLSX

WRITEBACKJOURNALDRAFT_Test_Cases.xlsx